TradeSmart API Latency Benchmark: Deployment and Handover Guide

tsolc.tradesmartonline.in

Ubuntu 24.04 on AWS

For the Linux system team

November 2026

Contents

1. What this is

2. How the static IP works

3. Prerequisites

4. Network and firewall

5. Install from GitHub

6. First-time app setup

7. Roles and users

8. Operations

9. Security notes

10. Troubleshooting

11. Handover acceptance

12. File reference

Audience: the Linux system team. No knowledge of the app is assumed.

Who does what. Throughout this guide:

- SYSTEM TEAM marks work for the system team.
- OWNER marks information or actions only the app owner can supply.
- APP ADMIN marks steps done in the app's web pages by a user listed in `TS_ADMIN_USERS`.

Replace everything in `<angle brackets>` with your own value. Never put real credentials in tickets, chat or this file.

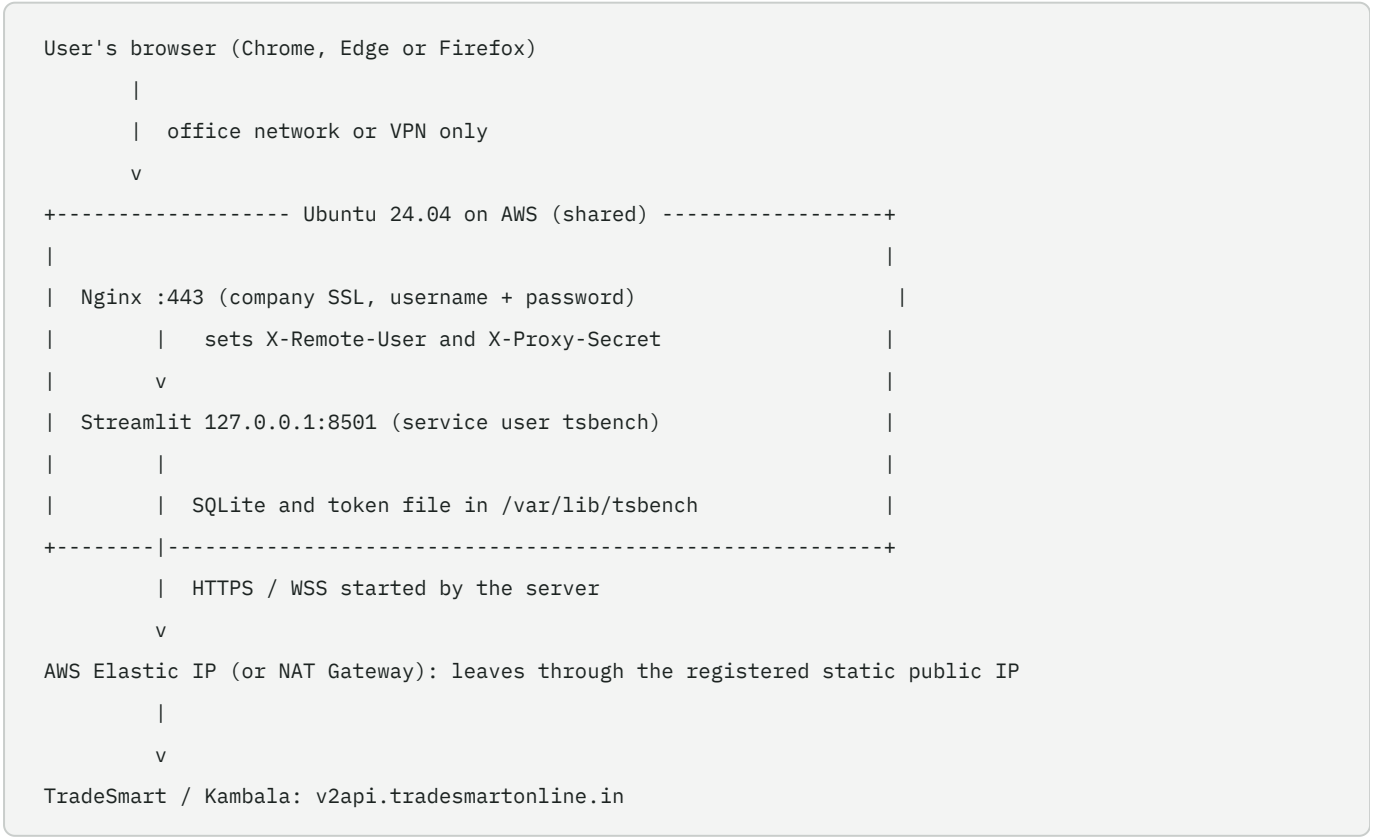
---

1. What this is

A Python (Streamlit) web app for internal use, served at `https://tsolc.tradesmartonline.in`. It measures how long TradeSmart API V2 requests take as seen from the server, stores every sample in SQLite, and reports P50, P95 and P99 per endpoint.

Automated benchmarks and endpoint checks only send read-only requests. One opt-in page, **Order latency test**, places **real orders on a live trading account** and cancels them straight away. Its limits: BUY LIMIT, quantity 1, priced below the market, at most ₹5,000 per order and 1,000 orders per IST day, restricted to named users.

All users share one TradeSmart account. Only one benchmark or order batch can run at a time.



2. How the static IP works

Requirement. The owner has a static IP registered with TradeSmart. TradeSmart's system must see that address on this app's traffic. This guide makes no claim about TradeSmart's or any regulator's rules. Questions about what must be registered, and when, go to the owner and TradeSmart.

What TradeSmart sees.

Traffic	Starts from	Address TradeSmart / Kambala sees
REST API calls	Server	Server's public outbound IP
WebSocket	Server	Server's public outbound IP
Script master downloads	Server	Server's public outbound IP
Token exchange (`GenAcsTok`)	Server	Server's public outbound IP
TradeSmart OAuth login page	**User's browser**	**The user's device or office IP**

- A user's device does not change the address on API traffic. Users talk only to Nginx; the server makes the API calls.
- The IP is the network source address. It is not a field in any request, and the app sends no forwarding headers.
- The OAuth login page is the single exception, because the admin opens it in their own browser. That step shows the device's IP. No other step does.

What the network must do. The server's outbound traffic to `v2api.tradesmartonline.in` must leave through the registered static IP. Use one of these:

Setup	What to configure
EC2 instance in a public subnet	Attach an **Elastic IP** to the instance. That address is the one to register. Leave `TS_OUTBOUND_BIND_IP` empty.
EC2 instance in a private subnet	Traffic leaves through the **NAT Gateway's Elastic IP**. That address is the one to register, and other instances behind the same gateway share it. Leave `TS_OUTBOUND_BIND_IP` empty.
Instance with several network interfaces	Set `TS_OUTBOUND_BIND_IP` to the **private address of the interface** that has the Elastic IP. It must appear in `ip -brief addr` . An Elastic IP itself never appears there, so don't use it here; connections would fail with "Cannot assign requested address".

Do not set `HTTP_PROXY` or `HTTPS_PROXY` in the service environment unless that proxy's public address is the registered IP. A proxy changes the source address that TradeSmart sees, and it also changes the measured latency.

How the app verifies it. The app asks an IP-check service what address its traffic comes from (`TS_IP_CHECK_URL` , default `https://api.ipify.org`) and compares the answer with the configured IP. The configured IP is the value an admin saved in Settings → Server IP, or `TS_STATIC_IP` if none was saved.

- Orders are refused if the IP is not set, cannot be detected, or does not match. This is on by default in server mode (`TS_ENFORCE_STATIC_IP`). Order batches run their own fresh check; the status shown on pages refreshes about every 5 minutes.
- Read-only requests only show a warning.

Limitation of this check. It proves where traffic to the *IP-check service* comes from, not traffic to TradeSmart. With policy routing, the two could leave differently, so the check could pass while TradeSmart sees another address. The app cannot ask TradeSmart what address it saw.

Verify on the server. SYSTEM TEAM

```
# The public IP seen by an outside service. It must equal the registered static IP.
curl -4 -s https://api.ipify.org; echo

# With TS_OUTBOUND_BIND_IP set, test from that address.
curl -4 -s --interface <local-bind-ip> https://api.ipify.org; echo

# Does TradeSmart's hostname publish IPv6 addresses, and does the server have global IPv6?
getent ahosts v2api.tradesmartonline.in
ip -6 route show default
```

If both of the last two commands show IPv6, API traffic may leave over IPv6 while the IP check uses IPv4. Ask the network team to resolve this before go-live.

If your VPC uses policy routing, confirm that both destinations leave the same way:

```
ip route get "$(getent ahostsv4 v2api.tradesmartonline.in | awk 'NR==1{print $1}')"
ip route get "$(getent ahostsv4 api.ipify.org | awk 'NR==1{print $1}')"
```

These commands show only the route on this server. Also confirm in the VPC route tables that both destinations leave through the same Elastic IP or NAT Gateway.

---

3. Prerequisites

Applications to install

The app is **Python only**. **PHP, Java, Node.js and a database server are not needed.** It stores its data in a local SQLite file. `install.sh` installs all of this, so nothing needs installing by hand:

Package	Why
`python3.12-venv`, `python3-pip`	Run the app in its own virtual environment. Ubuntu 24.04 includes Python 3.12.
`git`	Download the code from GitHub
`nginx`	Web server: SSL, login and access control. Already on your server, and your other sites are not changed.
`apache2-utils`	`htpasswd`, for login accounts
`sqlite3`	Database backups and restores
`openssl`, `curl`	Generate the proxy secret; run health checks

Port `8501` on `127.0.0.1` must be free. The installer stops if another program uses it.

What each person provides

Item	Requirement	Provided by
Server	Ubuntu 24.04 on AWS. Suggested spare capacity: 2 vCPU, 4 GB RAM, 20 GB disk. This is a suggestion, not a measured requirement.	System team
Static public outbound IP	Elastic IP (section 2). The value may arrive after install; orders stay blocked until it is set.	System team, registered with TradeSmart by the owner
Code folder	`/vns-work/code/tsolc.tradesmartonline.in` : empty, or not yet created	System team
DNS	`tsolc.tradesmartonline.in` pointing to this server, reachable from the office network and VPN	System team
SSL certificate	Company certificate (full chain) and private key for `tsolc.tradesmartonline.in`	System team
Allowed networks	Office and VPN CIDRs, for the security group and the Nginx site file	Owner and system team
GitHub access	Read-only deploy key added to the private repo	Key generated by system team, added by owner
TradeSmart App Key	Value for `TS_APP_KEY` (secret)	Owner
TradeSmart Secret Key	Value for `TS_SECRET_KEY` (secret)	Owner
TradeSmart Client ID	Value for `TS_CLIENT_ID` (secret)	Owner
Redirect URL	`https://tsolc.tradesmartonline.in/callback` , registered in the TradeSmart API app	Owner
Admin usernames	Value for `TS_ADMIN_USERS`	Owner
Order usernames	Value for `TS_ORDER_USERS`	Owner
Login accounts	Created with `manage-user.sh`	System team

4. Network and firewall

Configure these in the instance's **AWS security group**. The kit never changes UFW or any firewall.

Inbound

Port	Protocol	From	Purpose
443	TCP	Office and VPN CIDRs	The app (HTTPS and WebSocket through Nginx)
80	TCP	Office and VPN CIDRs	TradeSmart `/callback` and redirect to HTTPS
22	TCP	Admin hosts only	SSH

Do not open port 8501. Streamlit listens on 127.0.0.1 only.

The site file also applies the office/VPN list on port 443. Port 80 has no list because it only sends redirects, so the security group is what limits port 80. If other sites on this server need 80 or 443 open more widely, the `allow` list in the site file still protects this app on 443.

Outbound

Destination	Port	When	Purpose
`v2api.tradesmartonline.in`	443	Always	REST, WebSocket (WSS), script master, token exchange. **Must leave through the static IP.**
`api.ipify.org` (or your `TS_IP_CHECK_URL`)	443	Always	Outbound IP check. If blocked, orders are refused.
`github.com`	22	Install and update	Code download with the deploy key
`pypi.org`, `files.pythonhosted.org`	443	Install and update	Python packages
Ubuntu mirrors	80, 443	Install and patching	OS packages
DNS and NTP	53, 123	Always	Name lookup and correct time

Load balancer

This guide assumes browsers reach Nginx directly. **If an AWS load balancer sits in front of Nginx, tell the owner first.** The `allow` rules and the rate limit would see the load balancer's address instead of the user's, so the site file needs changing.

---

5. Install from GitHub

Server layout

Item	Path
Service user and group	`tsbench`
Application code (Git checkout)	`/vns-work/code/tsolc.tradesmartonline.in`
Python virtual environment	`/opt/tsbench/venv`
Configuration	`/etc/tsbench/tsbench.env` (root:tsbench, 0640)
Data	`/var/lib/tsbench` (`benchmark.db`, `scriptmaster/`, `.access_token`, `api_run.lock`)
Logs	`/var/log/tsbench`
Backups	`/var/backups/tsbench`
Nginx site	`/etc/nginx/sites-available/tsolc.tradesmartonline.in`
Nginx password file	`/etc/nginx/tsbench.htpasswd`
Nginx proxy secret	`/etc/nginx/tsbench-proxy-secret.conf`

Item	Path
SSL certificate and key	Your own paths, set in the Nginx site file

The code folder is not a document root. The app runs as a service on `127.0.0.1:8501` and Nginx forwards requests to it. Never add a `root` pointing at the code folder, or the source code could be downloaded from the website.

The installer makes the code folder `root:tsbench`, with no access for other users. The `tsbench` user must be able to enter its parent folders. If it can't, the installer stops and says what to do.

5.1 Create the deploy key SYSTEM TEAM

```
sudo apt-get update
sudo apt-get install -y git
sudo mkdir -p -m 700 /root/.ssh
sudo ssh-keygen -t ed25519 -N "" -C "tsbench-deploy@$(hostname)" -f
/root/.ssh/tsbench_deploy_key
sudo tee -a /root/.ssh/config >/dev/null <<'EOF'
Host github.com
    IdentityFile /root/.ssh/tsbench_deploy_key
    IdentitiesOnly yes
EOF
sudo chmod 600 /root/.ssh/config
sudo cat /root/.ssh/tsbench_deploy_key.pub
```

The kit's scripts look for the key at exactly this path. They use it for every later update.

Send the printed **public** key line to the owner. Never send the file without `.pub`.

5.2 Add the key to GitHub OWNER

1. Open `Yahska04/tradesmart-api-latency-benchmark` on GitHub.
2. Go to **Settings** → **Deploy keys** → **Add deploy key**.
3. Title it with the server name and paste the public key.
4. Leave **Allow write access** unticked.
5. Click **Add key**.

5.3 Test the key SYSTEM TEAM

Accept the host key only after comparing its fingerprint with the SSH fingerprints GitHub publishes in its documentation.

```
sudo ssh -T git@github.com
```

The expected reply says you authenticated successfully but that GitHub does not provide shell access. Its exit code is 1, which is normal here.

5.4 Run the installer SYSTEM TEAM

```
sudo git clone git@github.com:Yahska04/tradesmart-api-latency-benchmark.git /root/tsbench-bootstrap
sudo bash /root/tsbench-bootstrap/deploy/install.sh
sudo rm -rf /root/tsbench-bootstrap
```

Optional input	Meaning
`GIT_REF=<ref>`	Branch, tag or commit. First install defaults to `main`. A re-run keeps the installed code unless this is given.
`REPO_URL=<url>`	Repository URL. Default: the repo above
`DEPLOY_KEY=<path>`	Path of the GitHub key. Default: `/root/.ssh/tsbench_deploy_key`

The installer installs packages, creates the `tsbench` user and folders, clones the code, and installs Python packages from the hash-checked `requirements.lock`. It creates the env file and proxy secret, copies the Nginx site file into `sites-available`, installs the systemd units and enables the backup timer. It keeps existing configuration, secrets, logins, data and your edited site file.

It is safe on a shared server. It does not: enable or edit any Nginx site, remove the default site, create a certificate, or change the firewall.

The first run leaves the service stopped, because the settings are still blank. Its summary lists what's missing, which is normal.

If GitHub refuses the key, the installer prints a public key and exits with code 2. Add that key (step 5.2) and run it again.

5.5 Fill in the configuration SYSTEM TEAM with values from OWNER

```
sudo nano /etc/tsbench/tsbench.env
```

Variable	Value	Supplied by
`TS_DEPLOYMENT`	`server`	Leave as set by the template
`TS_ENVIRONMENT`	`live`	Leave as set by the template
`TS_APP_KEY`, `TS_SECRET_KEY`, `TS_CLIENT_ID`	From the TradeSmart API app	Owner
`TS_REDIRECT_URL`	`https://tsolc.tradesmartonline.in/callback`	Leave as set by the template
`TS_ADMIN_USERS`	Comma-separated login names, for example `alice,bob`	Owner
`TS_ORDER_USERS`	Comma-separated login names	Owner
`TS_AUTH_PROXY_SECRET`	Generated by `install.sh`, which copies it to Nginx	Leave as is (to change it, see section 9)
`TS_ENFORCE_STATIC_IP`	`1`	Leave as set by the template

Variable	Value	Supplied by
`TS_DB_PATH`	`/var/lib/tsbench/benchmark.db`	Leave as set by the template
`TS_LOG_PATH`	`/var/log/tsbench/app.log`	Leave as set by the template
`TS_TOKEN_FILE`	`/var/lib/tsbench/.access_token`	Leave as set by the template
`TS_STATIC_IP`	Leave empty; an admin enters it in Settings	App admin
`TS_OUTBOUND_BIND_IP`	Only for an instance with several interfaces (section 2)	System team

Use `KEY=value` on one line, with no spaces around `=`. `.env.example` in the repo explains each variable.

`TS_APP_KEY`, `TS_SECRET_KEY`, `TS_CLIENT_ID`, `TS_AUTH_PROXY_SECRET` and `TS_ADMIN_USERS` must all be set, or the kit's scripts won't start or restart the service.

5.6 Create login accounts SYSTEM TEAM

Run this once per user. It asks for the password, and also changes the password of an existing user. Use exactly the names listed in `TS_ADMIN_USERS` and `TS_ORDER_USERS`. Names may use letters, digits, dot, underscore and dash.

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh add <username>
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh list
```

5.7 Configure and enable the Nginx site SYSTEM TEAM

The virtual host is in the repo at `deploy/tsolc.tradesmartonline.in.conf`, and the installer has copied it to `/etc/nginx/sites-available/tsolc.tradesmartonline.in`. Edit that copy:

```
sudo nano /etc/nginx/sites-available/tsolc.tradesmartonline.in
```

Replace every `CHANGE_ME` value:

Placeholder	Replace with
`CHANGE_ME_SSL_CERTIFICATE_PATH`	Path of the company certificate (full chain, server certificate first)
`CHANGE_ME_SSL_PRIVATE_KEY_PATH`	Path of its private key
`CHANGE_ME_OFFICE_CIDR`	Office network, for example `203.0.113.0/24`
`CHANGE_ME_VPN_CIDR`	VPN network. Add one `allow` line per extra network; delete this line if there is no VPN.

You may apply your own standards to the rest (logging, SSL ciphers, HTTP/2). **Keep these, or the app stops working:**

- `proxy_set_header Upgrade` and `Connection` (WebSocket)
- `proxy_set_header Host $http_host`
- `auth_basic`, `proxy_set_header X-Remote-User $remote_user` and the `include` of `tsbench-proxy-secret.conf`

- `proxy_set_header Authorization ""`, `proxy_read_timeout`, `proxy_send_timeout` and `proxy_buffering off`
- `allow 127.0.0.1/32`, which `healthcheck.sh` uses

Do not add a `root`. Then enable it:

```
sudo ln -s /etc/nginx/sites-available/tsolc.tradesmartonline.in /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

The installer and `update.sh` never overwrite your edited copy. If a later update changes the repo's file, `update.sh` says so and you compare the two yourself.

5.8 Start the service and check it SYSTEM TEAM

```
sudo systemctl start tsbench
sudo systemctl status tsbench --no-pager
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/healthcheck.sh
```

`healthcheck.sh` checks both services, the health endpoint and the site file. It also confirms Nginx asks for a login and ignores forged identity headers, and runs the self-check below. It exits with 0 only if everything passes.

To run the self-check alone, use the service's user and environment. It sends no API requests and no credentials, and it never prints a secret. Use this exact form:

```
sudo systemd-run --pipe --wait --collect -p User=tsbench -p
EnvironmentFile=/etc/tsbench/tsbench.env -p WorkingDirectory=/vns-
work/code/tsolc.tradesmartonline.in /opt/tsbench/venv/bin/python -m src.doctor
```

Before first-time app setup, these two results are expected:

- `[WARN] Access token: none yet`
- `[WARN] Outbound IP vs static IP` (the IP has not been set yet)

Every other line should be `PASS`. A `FAIL` must be fixed before handover.

---

6. First-time app setup

Do this in order. The token step needs the redirect URL registered first. The order test needs market hours.

6.1 Log in and enter the server IP APP ADMIN

1. Open `https://tsolc.tradesmartonline.in/` from the office network or VPN, and log in with your username and password.
2. Go to **Settings** → **Server IP**.
3. Enter the registered static public IP in **Server static IP** and click **Save IP**.

4. Confirm **Configured IP** and **Detected IP** match. Click **Check now** to re-test; otherwise the status refreshes about every 5 minutes. If they don't match, see Troubleshooting.

6.2 Register the redirect URL OWNER

Register this in the TradeSmart API app. The **Settings → Server IP** page shows the same value:

```
https://tsolc.tradesmartonline.in/callback
```

What happens: after login, TradeSmart sends the browser to `https://tsolc.tradesmartonline.in/callback?code=<one-time code>`. Nginx redirects that to `/Settings?code=<one-time code>`. If you're logged in as an admin, Settings shows **TradeSmart sent a login code**.

This works from the office network or VPN, because it's the admin's own browser that follows the redirect. If the redirect doesn't reach the app, use the paste fallback in section 6.3.

6.3 Generate the access token APP ADMIN

1. Go to **Settings → Get an access token**.
2. Click **1 · Open TradeSmart login**. The login page opens in a new tab in your browser. This step shows your device's IP (section 2).
3. Log in to TradeSmart. You return to Settings with the **TradeSmart sent a login code** box.
4. Click **Get access token**. The app shows **Access token saved. All users now share it.**

Paste fallback. If you don't land on Settings, even if the browser shows a connection error, copy the full address from the browser bar. Then go back to **Settings → Get an access token**, paste it into **2 · Paste the redirect URL (or just the code)**, and click **Get access token**. A code works once and expires quickly. If the exchange fails, log in again to get a new code.

The token is saved in `/var/lib/tsbench/.access_token` (0600) and is shared by all users. It survives restarts.

Token expiry. Expect tokens to expire. The exact lifetime isn't stated in the documentation available to us. When API calls report session errors, an admin generates a new token with the same steps.

6.4 Run the self-check SYSTEM TEAM

```
sudo systemd-run --pipe --wait --collect -p User=tsbench -p
EnvironmentFile=/etc/tsbench/tsbench.env -p WorkingDirectory=/vns-
work/code/tsolc.tradesmartonline.in /opt/tsbench/venv/bin/python -m src.doctor
```

All lines should now be `PASS`, including `Outbound IP matches static IP`. A `WARN` on order users is acceptable only if no order users are wanted yet.

6.5 Run a small read-only benchmark APP ADMIN

1. Open **Run Benchmark**.
2. Select `UserDetails` and `Limits`.
3. Set iterations to 20, warm-up to 2, delay to 500 ms, with warm connections.
4. Review the plan, then click **Start benchmark**.

5. Open **Performance Analytics** and confirm the requests succeeded.

6.6 Run the first small live order test OWNER or order user

This places real orders on the live account. Do it during market hours (IST 09:15–15:30), with the owner present.

1. Open **Order latency test**. Only users in `TS_ORDER_USERS` can use it.
2. Search for a liquid NSE share priced so that one share is under ₹5,000.
3. Enter **16** orders. At 8 orders per burst, that makes **2 bursts**. Keep the default offset (3% below the market) and the default delay (3 s).
4. Type the confirmation exactly as shown: `PLACE 16 ORDERS`.
5. Wait for the batch to finish. There should be no red alert.
6. In TradeSmart's order book, confirm that all 16 orders show as cancelled and that none filled.

If the batch stops with a red alert, read the order number it names and check that order in TradeSmart.

---

7. Roles and users

Role	How it's set	Can do	Cannot do
Any logged-in user	Account in `/etc/nginx/tsbench.htpasswd`	View dashboards and history, run read-only benchmarks, endpoint checks and the WebSocket benchmark	Open Settings, edit the endpoint registry, load demo data, place orders
Order user	Also listed in `TS_ORDER_USERS`	All of the above, plus the Order latency test and its open-order check	Open Settings, unless also an admin
Admin	Also listed in `TS_ADMIN_USERS`	Everything a logged-in user can do, plus Settings (server IP, access token, script master, data), endpoint registry edits and demo data	Place orders, unless also an order user

The sidebar lists Settings and Order latency test for everyone. Those pages refuse users without the role.

The admin and order lists are separate. Put a person in both lists if they need both roles. List names are not case-sensitive, but Nginx logins are, so use one spelling everywhere.

Add a user SYSTEM TEAM

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh add <username>
```

To give a role, add the name to `TS_ADMIN_USERS` or `TS_ORDER_USERS` in `/etc/tsbench/tsbench.env`. Then check that no run is active (section 8.9) and restart:

```
sudo systemctl restart tsbench
```

Remove a user SYSTEM TEAM

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh remove <username>
```

Also remove the name from both role lists. Check that no run is active, then restart the service. The restart also ends any session the person still has open.

List users SYSTEM TEAM

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh list
```

---

8. Operations

8.1 Service commands

Check that no run is active before `stop` or `restart` (section 8.9).

```
sudo systemctl status tsbench --no-pager
sudo systemctl stop tsbench
sudo systemctl start tsbench
sudo systemctl restart tsbench
```

8.2 Logs

```
sudo journalctl -u tsbench -f
sudo journalctl -u tsbench --since "1 hour ago" --no-pager
sudo tail -f /var/log/tsbench/app.log
sudo tail -f /var/log/nginx/error.log
```

Secrets are masked in the app's logs.

Rotation. The app rotates `/var/log/tsbench/app.log` itself at 2 MB and keeps 3 old files. `journald` rotates the service journal under the server's own settings.

8.3 Update

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/update.sh
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/update.sh <branch-tag-or-commit>
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/healthcheck.sh
```

`update.sh` takes a database backup first, installs the code and packages, runs the self-check and restarts the service. If the new version crashes the self-check or doesn't come up healthy, it rolls back by itself.

Exit code	Meaning
0	Updated
1	Update failed; the previous version is running again, or nothing changed
2	Update and rollback both failed; check `journalctl -u tsbench`
3	A run is active, so nothing was changed. Try again when it finishes.

`FORCE=1` skips the run check and **kills a running batch**, and it also discards local changes. Avoid it during market hours.

If the output says the Nginx site file changed, compare it with your live copy (section 8.5).

8.4 Roll back

```
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/update.sh rollback
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/healthcheck.sh
```

This returns to the commit before the last update. It restores code and Python packages only. If the newer version changed the database, also restore the backup taken just before the update (section 8.6).

8.5 Change allowed networks or the Nginx site

Edit your live copy, then test and reload. Update the security group too (section 4).

```
sudo nano /etc/nginx/sites-available/tsolc.tradesmartonline.in
sudo nginx -t
sudo systemctl reload nginx
```

To see how your copy differs from the repo's file:

```
sudo diff /etc/nginx/sites-available/tsolc.tradesmartonline.in /vns-
work/code/tsolc.tradesmartonline.in/deploy/tsolc.tradesmartonline.in.conf
```

To change the code version, use `update.sh`. Re-running `install.sh` keeps the installed code and your site file.

8.6 Backups and restore

`tsbench-backup.timer` backs up the database daily at 19:30 IST to `/var/backups/tsbench/benchmark-YYYYMMDD-HHMM.db.gz` and deletes backups older than 30 days. It's safe while the app is running. The token file and `/etc/tsbench/tsbench.env` aren't included, so back up the env file separately in your secrets store. Copy backups off the server if you need them to survive its loss.

```
sudo systemctl list-timers tsbench-backup.timer --no-pager
sudo systemctl start tsbench-backup
sudo journalctl -u tsbench-backup -n 5 --no-pager
sudo ls -lt /var/backups/tsbench | head
```

Restore:

```
sudo systemctl stop tsbench
sudo -u tsbench cp -p /var/lib/tsbench/benchmark.db /var/lib/tsbench/benchmark.db.before-restore
sudo rm -f /var/lib/tsbench/benchmark.db-wal /var/lib/tsbench/benchmark.db-shm
sudo bash -c 'gunzip -c /var/backups/tsbench/<backup-file>.db.gz >
/var/lib/tsbench/benchmark.db'
sudo chown tsbench:tsbench /var/lib/tsbench/benchmark.db
sudo chmod 0640 /var/lib/tsbench/benchmark.db
sudo -u tsbench sqlite3 /var/lib/tsbench/benchmark.db "PRAGMA integrity_check;"
sudo systemctl start tsbench
sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/healthcheck.sh
```

The integrity check must print `ok`. The restore also brings back the server IP saved at backup time, so confirm it in Settings.

After a restore, check the TradeSmart order book. A restored database doesn't know about test orders or results saved after the backup. That has two effects:

- The startup check may miss journalled orders. It still finds open orders tagged `latency-test` in today's order book.
- The daily order count can read low for that day, so don't rely on the cap until the next IST day.

8.7 Monitoring suggestions

Check	Command or target	Healthy result
App process	`curl -fsS http://127.0.0.1:8501/_stcore/health` (on the server)	`ok`
Full stack	`sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/healthcheck.sh`	Exit code 0
Service state	`systemctl is-active tsbench`	`active`
Certificate	`sudo openssl x509 -in <your-certificate-path> -noout -enddate`	Date is well in the future
Disk	`df -h /var/lib/tsbench /var/backups/tsbench`	Space available
Backup timer	`systemctl list-timers tsbench-backup.timer`	Next run listed

`healthcheck.sh` also fails if the self-check reports a `FAIL`, including a static IP mismatch or an unconfirmed test order. Treat those alerts as urgent.

8.8 App behaviours to know

Daily order cap. At most 1,000 PlaceOrder attempts per IST day, across all users. It is counted from saved results, and it resets by itself at midnight IST. There's no manual reset. Don't edit the database to change it.

Script master (instrument lists). The app downloads public lists from `https://v2api.tradesmartonline.in/<EXCH>_symbols.txt.zip` for NSE, NFO, BSE, BFO, BCD, CDS and MCX, without credentials, into `/var/lib/tsbench/scriptmaster/`.

- The download runs in the background once a page is opened after a start, and is re-checked hourly while the app is in use. Each exchange downloads at most once per IST day.

- A failed exchange keeps its previous list and is tried again at the next check.
- An admin can force it at **Settings → Script master → Refresh script master**.

One run at a time. A server-wide lock (`/var/lib/tsbench/api_run.lock`) allows one benchmark or order batch at a time. The OS releases it if the process dies. Never delete this file.

Restart and open-order recovery.

- On a clean stop, the app stops placing orders and gives the running burst up to 25 seconds to cancel what it placed. systemd allows 60 seconds before it kills the process.
- Each start runs `python -m src.recover_orders` first (`ExecStartPre` in the unit). It looks for test orders left open by a crash or power cut, and cancels them. It touches only orders in the app's own journal or tagged `latency-test` . It never cancels any other order.
- **The service starts even if this check fails or reports a problem** (the unit uses `ExecStartPre=-`). A running service therefore doesn't prove the check was clean. Read the journal after every restart (section 8.9).
- New order batches are refused while any test order is unconfirmed, and after a placement that got no reply, until the open-order check has read the order book.
- Finding an order that got no reply relies on the order book returning the `remarks` tag. The API documentation doesn't list that field for certain, so **confirm it during the first live test** (section 11) and check the order book by hand after any no-reply alert.

When `ATTENTION` appears (in `journalctl -u tsbench` , or when you run the command below):

Message	Meaning	What to do
`ATTENTION: N test order(s) unconfirmed, but no access token is available`	Orders could not be checked	Check the order book now. An admin generates a token, then run the command below.
`ATTENTION: check order <number> in TradeSmart`	The order filled, or could not be confirmed closed. Expired tokens and network failures also cause this.	Tell the owner at once. Check the order in TradeSmart and cancel it by hand if it's open.
A summary showing `FILLED` above 0	A real position now exists on the account. Each fill is reported only once.	Save the order number. The owner decides what to do with the position. The app doesn't close positions.

Check again after the fix. An order user can click through **Order latency test → Open test orders**, or you can run this with the service's environment:

```
sudo systemd-run --pipe --wait --collect -p User=tsbench -p
EnvironmentFile=/etc/tsbench/tsbench.env -p WorkingDirectory=/vns-
work/code/tsolc.tradesmartonline.in /opt/tsbench/venv/bin/python -m src.recover_orders
```

Exit code 0 means clean or nothing to do. Exit code 1 means someone must look at the printed lines. If it prints `Skipped` , a run holds the lock; try again when it ends.

8.9 Before a planned restart or reboot

Do this before `systemctl restart` , OS patching or a reboot. `update.sh` checks by itself, but a reboot won't:

```
sudo -u tsbench flock -n /var/lib/tsbench/api_run.lock true && echo "No run active" || echo "A
run is active"
sudo cat /var/lib/tsbench/api_run.lock.info 2>/dev/null
```

If a run is active, the second line shows who started it. Wait for the run to end or ask that person to press **Stop**. Prefer outside market hours (IST 09:15–15:30) for planned work. After the restart, check the journal for `ATTENTION`:

```
sudo journalctl -u tsbench -b --no-pager | grep -E "ATTENTION|Open-order check"
```

---

9. Security notes

Secrets and permissions.

Secret	Location	Owner and mode
TradeSmart keys, client ID, proxy secret	`/etc/tsbench/tsbench.env`	root:tsbench, 0640
Shared access token	`/var/lib/tsbench/.access_token`	tsbench, 0600
Proxy secret (Nginx side)	`/etc/nginx/tsbench-proxy-secret.conf`	root:root, 0640
User passwords (bcrypt hashes)	`/etc/nginx/tsbench.htpasswd`	root:www-data, 0640
SSL private key	Your path, per company standard	Per company standard
GitHub deploy key	`/root/.ssh/tsbench_deploy_key`	root, 0600

Never commit `.env` or any file holding real values. `.gitignore` excludes `.env`. Database backups hold no token or keys, but they do hold test results and order numbers, so keep them restricted (`/var/backups/tsbench` is tsbench, 0750).

Proxy secret. Streamlit trusts `X-Remote-User` only if `X-Proxy-Secret` matches `TS_AUTH_PROXY_SECRET`. This stops any other process on the server from posing as a user by calling port 8501 directly. If the secret is missing or wrong, the app denies access.

Identity headers. Nginx overwrites `X-Remote-User` and `X-Proxy-Secret` on every request, so a value sent by the browser is discarded. `healthcheck.sh` tests this. Nginx also strips the password (`Authorization`) before passing requests to the app. If you change the Nginx site file, keep this behaviour.

Network exposure. Streamlit binds to 127.0.0.1:8501. Don't change that or open the port.

Code folder. It holds source code and must never be served as files. The site file has no `root`, and the code folder isn't readable by other users. Nginx doesn't need to read it.

Basic auth.

- The browser sends the password with each request, so it relies on TLS. Never serve the app over plain HTTP.
- There's no logout. The browser remembers the login until it closes.
- There's no lockout or multi-factor login. Nginx rate-limits requests per client IP (10 per second, burst 200), which only slows guessing. Rely on the network restriction, and consider `fail2ban` on the Nginx

log.
- **Safari** may not send basic-auth credentials on WebSocket connections, which would stop the app from loading. Recommend Chrome, Edge or Firefox. If Safari matters to you, verify it during acceptance.

Rotating the proxy secret. Generate a value with `openssl rand -hex 32` and put it in `TS_AUTH_PROXY_SECRET` in `/etc/tsbench/tsbench.env`. Then run `sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/install.sh`, which copies it to Nginx, reloads Nginx and restarts the service.

OAuth callback. TradeSmart's documented login has no `state` value, so the app can't prove a `/Settings?code=` link started from its own login button. That's why it never exchanges a code automatically: an admin must press **Get access token**, and should do so only right after starting a login themselves.

Moving to SSO later. The app only reads the two headers, so no code change is needed. Put `oauth2-proxy` (or similar) in front through Nginx `auth_request`, set `X-Remote-User` from the SSO user name, and keep sending `X-Proxy-Secret`. Update `TS_ADMIN_USERS` and `TS_ORDER_USERS` to the SSO names.

---

10. Troubleshooting

Symptom	Likely cause	Fix
Login box keeps coming back (401 loop)	Wrong password, user not in the password file, or Nginx can't read the file	`sudo bash /vns-work/code/tsolc.tradesmartonline.in/deploy/manage-user.sh list`. Reset the password. Check `/var/log/nginx/error.log`.
403 from Nginx	Client address is not in the `allow` list of the site file	Check the client's VPN. Change the allowed networks (section 8.5).
429 from Nginx	Request rate limit reached, or many users share one source address	Wait and reload. If it recurs, raise the limit in the site file (section 8.5).
"Access denied" and "did not come through the login proxy"	`X-Proxy-Secret` doesn't match `TS_AUTH_PROXY_SECRET`, or someone is calling port 8501 directly	Run `install.sh` again, which copies the env value to Nginx and reloads it.
"Access denied", `TS_AUTH_PROXY_SECRET` not configured	The variable is empty in the env file	Run `install.sh` again, which generates one.
"The login proxy did not identify a user"	Nginx isn't sending `X-Remote-User`	Restore the required lines in the site file (step 5.7).
"Connection error" banner, or the page spins forever	WebSocket `/_stcore/stream` fails through Nginx. Often Safari, an intermediate proxy or a changed Nginx config.	Try Chrome, Edge or Firefox. Check the required lines in the site file (step 5.7). Check `/var/log/nginx/error.log`.
Redirect after TradeSmart login shows a connection error	The browser can't reach `tsolc.tradesmartonline.in`, often because the VPN is off	Connect to the VPN, or use the paste fallback (section 6.3).
`nginx -t` fails on `CHANGE_ME`	Placeholders are still in the site file	Replace them (step 5.7).

Symptom	Likely cause	Fix
`nginx -t` fails with a duplicate variable or zone	Another site defines the same name. The kit's names all start with `tsolc_`, so check any changes you made.	Rename the duplicate.
The other site's page appears at this address	DNS or `server_name` doesn't match, or the site isn't enabled	Check DNS and `sudo nginx -T | grep tsolc`.
Installer stops: port 8501 is already used	Another program on this shared server uses it	Free the port. To use another, the unit, site file and `lib.sh` must all change together; ask the owner.
Installer stops: `tsbench` can't read the code folder	A parent folder blocks it	Allow entry, for example `sudo chmod o+x /vns-work /vns-work/code`.
502 Bad Gateway	Streamlit isn't running	`sudo systemctl status tsbench`. `sudo journalctl -u tsbench -n 100 --no-pager`. `curl -fsS http://127.0.0.1:8501/_stcore/health`.
Orders blocked: static IP "has not been set yet"	Nobody has entered the IP	APP ADMIN Settings → Server IP.
Orders blocked: traffic "leaves from X, but the static IP is set to Y"	Routing or NAT is wrong, or the saved IP is wrong	Compare with `curl -4 -s https://api.ipify.org`. Fix the NAT or correct the IP (section 2). Do not turn off enforcement to get around this.
Orders blocked: "Couldn't detect this server's public IP"	Outbound to the IP-check service is blocked	`curl -4 -sv https://api.ipify.org`. Open outbound 443 (section 4). Orders stay blocked until this works.
Script master failed	Outbound blocked to `v2api.tradesmartonline.in`, or a disk or permission problem	`curl -sI https://v2api.tradesmartonline.in/NSE_symbols.txt.zip`. Check `/var/lib/tsbench/scriptmaster/` is owned by `tsbench`. Retry in Settings → Script master. Earlier lists stay in use.
API calls report session or token errors	Access token expired	APP ADMIN Generate a new token (section 6.3).
"No token received" in Settings	The one-time code was used or expired, or the keys are wrong	Log in to TradeSmart again for a new code. Check `TS_APP_KEY` and `TS_SECRET_KEY`.
"database is locked" or "Permission denied" on a data file	Wrong owner, often after running something as root, or another process holds the DB	`sudo chown -R tsbench:tsbench /var/lib/tsbench`, then `sudo chmod 600 /var/lib/tsbench/.access_token`. Run commands with the `systemd-run` form shown in this guide.
"A run started by X is running"	Another benchmark or batch is running. Only one runs at a time.	Wait, or ask X to stop it. Don't delete the lock file. If the process died, the lock has already been released.
New order batch refused because test orders are unconfirmed	An earlier order couldn't be confirmed closed	See `ATTENTION` in section 8.8. Use **Order latency test** → **Open test orders**.
`ATTENTION` in the journal, but the service is running	`src.recover_orders` reported a problem. By design, this doesn't stop the service.	See section 8.8.

Symptom	Likely cause	Fix
Service fails to start at the `ExecStartPre` stage	A failure of `src.recover_orders` itself is ignored, so look at the setup: a missing `/etc/tsbench/tsbench.env`, `tsbench` user, `/vns-work/code/tsolc.tradesmartonline.in`, or a folder in `ReadWritePaths`. A very slow check can also hit the 180 s start timeout.	`sudo journalctl -u tsbench -b --no-pager`, and look for `Failed at step` or `Failed to load environment files`. Fix it, or run `install.sh` again.
Service stops retrying ("start request repeated too quickly")	More than 10 failed starts in 5 minutes	Fix the cause, then `sudo systemctl reset-failed tsbench && sudo systemctl start tsbench`.
`update.sh` exits with code 3	A run is active	Try again when the run ends.
"Read-only file system" errors in the journal	`ProtectSystem` in the unit allows writes only to the data and log folders, and a path setting points elsewhere	Keep `TS_DB_PATH`, `TS_LOG_PATH` and `TS_TOKEN_FILE` under `/var/lib/tsbench` and `/var/log/tsbench`. Don't loosen the unit.

11. Handover acceptance

Checklist

System team

- [ ] Server meets section 3, and the security group matches section 4
- [ ] Other sites on this server still work
- [ ] No `CHANGE_ME` left in the Nginx site file, and it has no `root`
- [ ] `https://tsolc.tradesmartonline.in/deploy/install.sh` does not return a file
- [ ] After `curl -4 -s https://api.ipify.org` matches, confirm TradeSmart traffic takes the same route
- [ ] `curl -4 -s https://api.ipify.org` returns the registered static IP
- [ ] VPC routing confirmed for `v2api.tradesmartonline.in` and `api.ipify.org`
- [ ] `/etc/tsbench/tsbench.env` is filled in, owned root:tsbench, mode 0640
- [ ] Company SSL certificate installed; the browser shows no warning
- [ ] Login works from the office network and from VPN
- [ ] Access from outside the allowed networks is refused
- [ ] `curl http://<server>:8501` from another host fails
- [ ] `healthcheck.sh` passes
- [ ] `src.doctor` shows no `FAIL`
- [ ] A backup ran, and a restore was tested once
- [ ] `sudo systemctl restart tsbench` works, and the journal shows the open-order check
- [ ] A server reboot brings the service back without help
- [ ] `update.sh` tested once

Owner and app admin

- [ ] Admin can open Settings. A non-admin cannot.
- [ ] Order user can open the Order latency test. Others cannot.
- [ ] Server IP saved, and the app shows it matches
- [ ] Redirect URL registered with TradeSmart, and token generated
- [ ] Read-only benchmark completed
- [ ] 16-order live test completed, and all orders show cancelled in the order book
- [ ] Confirmed with the owner: TradeSmart's order book returns the `remarks` value `latency-test` on test orders
- [ ] Each user's browser verified to load the app (Safari only if needed)

Tested vs not tested

Tested, off Ubuntu:

- The automated test suite on Python 3.12 and 3.13, with a mocked network and no real TradeSmart calls.
- `deploy/tsolc.tradesmartonline.in.conf` on Nginx 1.24.0 (Ubuntu 24.04's version) running on Windows, with a test certificate and plain-text test passwords, beside a second site defining the common `$connection_upgrade` name.
- A full rehearsal through that Nginx against a mock TradeSmart: role checks, entering the static IP in Settings, the login code reaching the token exchange intact, a 16-order batch, an IP change refusing orders at send time, no user or proxy headers reaching the API, and a hard kill mid-burst with 8 orders open, all cancelled by `src.recover_orders`.
- ShellCheck and `bash -n` on all scripts. `requirements.lock` resolves to Linux wheels with hashes.
- `install.sh`, `update.sh` (including auto-rollback) and `manage-user.sh` only in a sandbox with stubbed system commands, which also confirmed other Nginx sites are left alone.

Not tested, because no Ubuntu machine was available:

- A real install: `apt`, `systemd` units and hardening, `systemd-run`, `flock`, bcrypt logins, `healthcheck.sh`, and the deploy key against GitHub.
- Clean shutdown on `SIGTERM`. The open-order check at the next start is the backstop for this.
- Your AWS networking, the company certificate, and any request against the live TradeSmart API from this server.

Treat the first install as validation, and report anything that differs from this guide to the owner.

---

12. File reference

Path	What it is
`deploy/tsolc.tradesmartonline.in.conf`	**Nginx virtual host.** The system team adds the certificate paths and allowed networks.
`deploy/install.sh`	Install or re-apply. Optional inputs: `GIT_REF`, `REPO_URL`, `DEPLOY_KEY`
`deploy/update.sh [ref|rollback]`	Update or roll back; automatic backup and rollback
`deploy/backup.sh`	Database backup, run through `tsbench-backup.service`

Path	What it is
`deploy/manage-user.sh add|remove <username>`, `list`	Manage Nginx logins
`deploy/healthcheck.sh`	Check services, Nginx login, forged headers and the self-check
`deploy/lib.sh`	Shared paths and helpers for the scripts
`requirements.lock`	Hash-checked Python packages installed on the server
`requirements.txt`, `requirements-dev.txt`	Runtime packages; runtime plus test tools for developers
`deploy/tsbench.service`	Main systemd unit, with the `ExecStartPre` open-order check
`deploy/tsbench-backup.service`, `deploy/tsbench-backup.timer`	Scheduled backup
`deploy/tsbench.env.example`	Template for `/etc/tsbench/tsbench.env`
`.env.example`	Explains every variable (local and server)
`src/doctor.py`	Self-check: `python -m src.doctor`
`src/recover_orders.py`	Open-order check: `python -m src.recover_orders`
`src/access.py`	Login header check and roles
`src/network_guard.py`	Static-IP check
`src/outbound.py`	Optional source address
`src/token_store.py`	Shared token file
`src/run_lock.py`	One-run-at-a-time lock
`src/order_test.py`	The only code that sends orders
`config/endpoints.json`	Endpoint registry and script master URLs
`docs/ARCHITECTURE.md`	Developer reference
`README.md`	Overview and local PC setup